UFUG2106 Final Project: Truth Table to Circuit

Task

In this project, you will solve a **logic synthesis** problem: constructing a combinational circuit from a truth table. Given a truth table with n inputs and m outputs, your program must construct an equivalent circuit using **2-input AND**, **2-input OR**, and **1-input NOT** gates. Each test case specifies the area cost and delay of each gate type, together with a **target delay** for the circuit. Your goal is to produce a correct circuit with small area while meeting the target delay.

A circuit must be acyclic. A gate's output may be used by multiple gates or circuit outputs. Submit **one circuit for each test case**. The circuit must implement **all m outputs correctly**.

Inputs

For each test case, your program receives a plain-text file with the following lines. The first seven lines are required; the last two are optional. Write only the values, without field names.

Line	Contents	Description
1	`n`	Number of inputs, $1 \le n \le 18$.
2	`m`	Number of outputs, $m \ge 1$.
3	`truth_table`	2^n groups on one line, separated by spaces; each group has m bits.
4	`area delay`	Area and delay of a 2-input AND gate (`AND2`).
5	`area delay`	Area and delay of a 2-input OR gate (`OR2`).
6	`area delay`	Area and delay of a 1-input NOT gate (`NOT`).
7	`target_delay`	Target circuit delay.
8	`runtime_limit`	Optional program runtime limit in milliseconds.
9	`ram_limit`	Optional program memory limit in KB (1 KB = 1024 bytes).

For example, the following input has three inputs and two outputs, with its truth table shown on the right.

```
3
2
00 00 10 11 10 11 10 11
2 2
8 3
1 1
5
```

x_0	x_1	x_2	y_0	y_1
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	1
1	0	0	1	0
1	0	1	1	1
1	1	0	1	0
1	1	1	1	1

The truth-table line lists one group of m output bits per input assignment. Separate groups with spaces; do not insert spaces within a group. All indices start at zero:

- **Input index i ($0 \le i < 2^n$):** The assignment is the n-bit binary representation of i, with x_0 as the most significant bit. Group i contains its output bits.
- **Output index j ($0 \le j < m$):** Character j from the left within group i gives output y_j for assignment i. Each character is `0` or `1`.

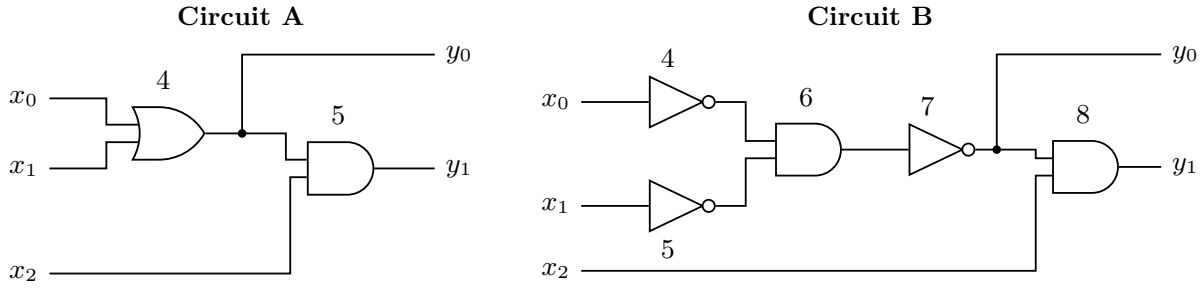
For example, group 2 is `10`, mapping input $(0, 1, 0)$ to outputs $(1, 0)$.

Each gate line contains two space-separated positive integers: `area` and `delay`. Target delay is also a positive integer, using the same abstract time unit as gate delay. Each gate type may be used any number of times.

Limits are nonnegative integers; **0 or an omitted trailing line means no limit**. For a RAM limit only, write 0 on line 8 and the RAM limit on line 9.

Outputs

The two circuits below implement the Inputs example. A dot marks a shared connection.



For each input file, write one plain-text output file. Each line except the last defines one gate; the last line lists the signal IDs connected to the m outputs. **Each gate line starts with its ID; no gate count or output label is written.**

Signal IDs are assigned as follows:

- **ID 0:** Constant 0.
- **IDs** $1, \ldots, n$**:** Inputs $x_0, \ldots, x_{n-1}$, respectively.
- **IDs** $n+1, n+2, \ldots$**:** Gate outputs, numbered consecutively in line order.

Line format	Description
`<ID> AND2 <FANIN0> <FANIN1>`	A 2-input AND gate.
`<ID> OR2 <FANIN0> <FANIN1>`	A 2-input OR gate.
`<ID> NOT <FANIN0>`	A 1-input NOT gate.
`<out0> ... <out(m-1)>`	Final line: exactly m signal IDs for $y_0, \ldots, y_{m-1}$.

Each gate line has the form `<ID> <GATE> <FANIN0> [<FANIN1>]`. Separate tokens with whitespace. Each gate may reference only smaller IDs. Output IDs may reference any defined gate, input, or constant 0. With no gates, write only the output line. Use `<ID> NOT 0` to generate constant 1.

For example, the following file represents **Circuit B**, where $n = 3$:

```
4 NOT 1
5 NOT 2
6 AND2 4 5
7 NOT 6
8 AND2 7 3
7 8
```

The first two lines define gates 4 and 5, computing $\neg x_0$ and $\neg x_1$. Gate 6 computes their AND; gate 7 inverts it to obtain $x_0 \vee x_1$. Gate 8 computes the AND of gate 7 and input 3 (x_2). The final line `7 8` connects y_0 to gate 7 and y_1 to gate 8.

Evaluation

- **Valid solution.** A test case passes only if the output follows the specified format, describes an acyclic combinational circuit using the allowed gates, and matches the truth table on **every output for all 2^n input assignments**. The program must also satisfy any positive `runtime_limit` and `ram_limit` specified for that case. **Otherwise, the case fails.**
- **Area.** The circuit area is $A(C) = \sum_{t \in \{\text{AND2,OR2,NOT}\}} N_t(C)\, a_t$, where $N_t(C)$ is the number of gates of type t listed in the output file, and a_t is that type's area cost. Each gate is counted once, even if reused. Inputs, constants, and wires have zero area.
- **Delay.** The circuit delay is the largest sum of gate delays on any path from an input or constant 0 to an output. Inputs, constant 0, and wires add no delay. **Exceeding the target delay incurs a score penalty.** A circuit that exceeds `target_delay` still passes if it meets all valid-solution requirements above.
- **Runtime.** Runtime is the elapsed wall-clock time **from program start to program exit**. It is measured per test case and excludes the evaluator's correctness checks.

For the circuits in the Outputs example, the target delay is 5:

Circuit	Gates used	Area	Delay
A	1 OR, 1 AND	$8 + 2 = 10$	$3 + 2 = 5$
B	3 NOT, 2 AND	$3 \times 1 + 2 \times 2 = 7$	$1 + 2 + 1 + 2 = 6$

Circuit A meets the target delay. Circuit B has smaller area but exceeds the target delay and incurs a penalty.

The score for a valid solution is based on area, delay, and runtime. In general, smaller area, lower delay, and shorter runtime lead to a higher score.

Project Timeline

Complete this project in **teams of two students**.

You may use **any programming language**; C++ may offer better runtime performance. Your program must run **sequentially on a CPU using a single thread**. Your program executable must accept an input file path and an output file path as command-line arguments:

```
<program_executable_path> <input_path> <output_path>
```

It must read the plain-text input from `input_path`, write the plain-text output to `output_path`, and then exit. The evaluator invokes this interface separately for each test case.

We will provide an evaluator binary for development. Run it with the path to your program executable and either a benchmark folder or a single case:

```
./evaluator <program_executable_path> <benchmark_folder>
./evaluator <program_executable_path> <benchmark_case>
```

The evaluator produces a CSV report for both single-case and folder runs. Passing requires valid output, functional correctness, and compliance with any positive runtime/memory limits. Each case is marked pass/fail. Area, delay, and runtime are reported **only for passed cases**.

Stage 1: Initial Method (2 weeks)

Deadline: **To Be Determined**. Simple cases will be released at the start of Stage 1. By the deadline, **submit your source code and a report**. The report must include:

- **Method:** Describe your algorithm and how it constructs circuits from the truth table using the given gate library.
- **Results:** Report pass/fail for every released case, and area, delay, and runtime for passed cases.

Stage 2: Method Improvement (2 weeks)

Deadline: **To Be Determined**. Medium-difficulty cases will be released at the start of Stage 2. By the deadline, **submit your source code and a report**. The report must include:

- **Method Improvement:** Explain the changes to your Stage 1 method and the reasons for them.
- **Results and Comparison:** Run both methods on **all cases released in both stages**. Compare pass/fail for every case and area, delay, and runtime for each method that passes it.
- **Individual Contributions and Teamwork:** Describe each member's contributions and how the team divided and coordinated the work.

Live Q&A

Date: **To Be Determined**. After the Stage 2 submission, **both team members must attend the live Q&A and answer questions collaboratively** based on the Stage 2 report and source code.

Final tests on the source code submitted in Stage 2 will contribute to your project grade.

Grading

Refer to the course syllabus.